HW04: A Bug's Life

- Assignment HW04 should be done alone.
- You should seek help completing this assignment from the TAs at the evening lab.

Learning Objectives

- Learn the impact a bug can have in the real world.
- Explore life path numbers.
- Practice creating your own fruitful functions.
- Learn to use modulus to capture remainders.

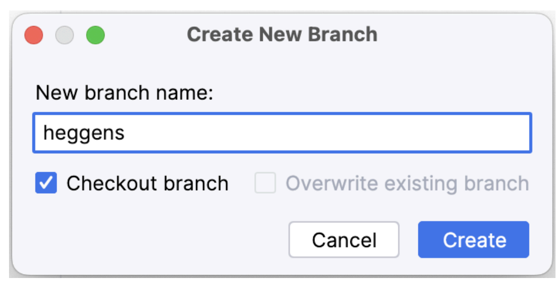
How to Start

- **To begin, make a copy of this document by going to File >> Make a Copy...**
- **Set the Share settings of this document to "Anyone with the link" as a Viewer.**

- **Change the file name of this document to *username* - HW04: A Bug's Life (for example, heggens - HW04: A Bug's Life). To do this, click the label in the top left corner of your browser.**
- **Next, go to the [Github classroom for HW04](#).**
- **Paste the link to your Github repo here:**

Github Repo Link:	https://github.com/Berea-College-CSC-226/hw04-a-bug-s-life-Antonio-MIT

- **Open PyCharm.**
- **Use Clone Repository to clone the HW04 repository using the link above.**
- **Create a new git branch, named with your username (e.g., heggens for me).**



Pair Programming

Pair programming is an agile software development technique from industry in which two programmers work as a pair together on one computer. The driver types code, while the other, the navigator, reviews each line of code as it is typed. The two programmers converse continually and switch roles frequently. It is what we want to aspire to practice in this course, and have been practicing in our teamwork assignments. Let's tie the practice to a little theory about *why* we do pair programming.

Task 1: Pair Programming, a Reading

Begin this assignment by reading this research article "The Costs and Benefits of Pair Programming" by Alistair Cockburn and Laurie Williams.

After you have read the article, respond to the following prompts. You need to write a paragraph of 3-5 sentences for each prompt. The goal is that your response shows you have thought about the article and added your own experience or insights to its ideas.

Go to **hw04_questions.md** and answer **SECTION 1**, section, then come back to this document to complete the rest of the assignment

---

In the remainder of this assignment, you will gain practice in creating and using a fruitful function which employs integer division and modulus.

The key learning in this activity is to use fruitful functions for returning values. Note that when a function returns a value, it is typical to put the value into a variable or use it directly. The following examples illustrate both of these ideas:

- **hw04_peel_digits.py** (check out the `peel_digits` function definition and call)
- **hw04_is_even.py** (check out the `is_even` function definition and call)

Modulus (Remainder) operator

The modulus operator, indicated by the percent symbol (`%`), calculates the remainder after dividing the number to the left with the number to the right. For example,

```
10 % 3 = 1
```

because 3 divides into 10 three times and leaves 1 as a remainder. If you were to take integer division and modulus operators together, you can undo division to get the original number back:

```
10 % 3 = 1

10 // 3 = 3
```

```
(3 * 3) + 1 = 10
```

You may find the following programs useful to your understanding of the modulus operator:

- **hw04_is_even.py** uses modulus to determine whether or not a number is even.
- **hw04_making_change.py** is an example that demonstrates how to use modulus and integer division to make change for a certain amount of money (in cents).

Life Path Numbers

Numerologists consider the Life Path Number to be the most important number in one's numerology chart. The Life Path Number is a number which is derived from the sum total of the digits that make up your birth date (with a couple of exceptions).

The exceptions occur with master numbers, such as the month of November, which is the 11th month, or birthdays on the 11th or 22nd day, or sum totals of 11, 22, or 33. These master numbers are not converted to single digits in the process of computing a Life Path number. Master numbers are always 11, 22, and 33 (*not* all numbers divisible by 11).

Steps to Calculating a Life Path Number

- **Step 0 - Ask the user to enter their birth year.**
- **Step 1 - Month: *Convert* the number of the birth month to digits and sum them, unless the month is November, in which case it is left as an 11.**
- **Step 2 - Day: *Convert* the birthday day to digits and sum these, unless the day is the 11th or the 22nd, in which case the 11 and 22 are left as an 11 or a 22.**
- **Step 3 - Year: Convert the year to 4 digits, and sum the four digits.**
- **Step 4 - Sum: Add all these digits together from Steps 1, 2, and 3. Reduce this total sum further by adding the remaining digits together until a single digit or a Master Number is obtained. Remember, Master Numbers are 11, 22, and 33.**

Example 1

A birthday of October 11, 1975 produces a Life Path Number of 7:

- **Step 1 - Month: Convert the number of the birth month to digits and sum them, unless the month is November, in which case it is left as an 11:**
  ```
  October => 10 => 1 + 0 = 1
  ```
- **Step 2 - Day: Convert the birthday day to digits and sum these unless the day is the 11th or the 22nd, in which case the 11 and 22 is left as an 11 or a 22:**
  ```
  11 is a master number, so it is not reduced
  ```
- **Step 3 - Year: Convert the year to 4 digits, and sum the four digits.**
  ```
  1975 => 1 + 9 + 7 + 5 = 22
  22 is a master number, so it is not reduced
  ```
- **Step 4 - Sum: Sum all these digits together from Steps 1, 2, and 3. Reduce this total sum further by adding the remaining digits together until a single digit is obtained.**

```
1 + 11 + 22 = 34
 Then 3 + 4 = 7
```

Example 2

A birthday of December 31, 1981 producing a Life Path Number of 8:

- **Step 1:** `December => 12 => 1 + 2 = 3`
- **Step 2:** `31 => 3 + 1 = 4`
- **Step 3:** `1981 => 1 + 9 + 8 + 1 = 19`
  `19 => 1 + 9 = 10`
  `10 => 1 + 0 = 1`
- **Step 4:** `3 + 4 + 1 = 8` **is the life path number.**

Example 3

A last example birthday of January 3, 2005 producing a Life Path Number of 11

- **Step 1:** `January => 1`
- **Step 2:** `3 => 3`
- **Step 3:** `2005 => 2 + 0 + 0 +5 = 7`
- **Step 4:** `1 + 3 + 7 = 11 => 11` **is a Master Number so it is not reduced. It is the Life Path Number.**

For an online Life Path Calculator see http://seventhlifepath.com/. You can use this to check your computations.

Your Task

Plan

First, plan out your algorithm. I've included a PLAN.md file to give you space for designing your algorithm. One key step to designing an algorithm is understanding and breaking down the problem. This is often known as functional decomposition. Using the PLAN.md file, detail how you will go about implementing the life path numbers program.

Commit and push your PLAN.md changes before you begin any coding!

Implement

Once you have a plan in place, create a new program for the life path number problem. In your program, you must include *at least* one fruitful function which does the following computation:

- [x] ~~Takes in an integer as an input parameter.~~
- [x] ~~Uses modulus (%) to determine if the input is a multiple of 11 (i.e. 11 or 22 or 33). If it is and is 33 or less, the input is a master number, so simply return the master number.~~
- [x] ~~If the input is not a master number, convert the input to digits and return the sum of the digits.~~

Also make sure your program includes the following:

- [x] ~~Uses function(s) for encapsulation with triple-quoted docstring for each of these functions. This docstring must include a description of the main purpose of the function, and descriptions of all input parameters as well as what is returned by the function.~~
- [x] ~~Create a `main()` function which takes in the user's input for month, day, and year, and prints the user's life path number.~~
- [x] ~~Call the function(s) created above to solve the life path number problem. Reuse the function as many times as is necessary.~~
- [x] ~~Makes use of the modulus function in determining if a number is a Master number.~~
- [x] ~~Do something creative and interesting while informing the user of their Life Path Number. This might be a graphical display of the Life Path Number and/or a description of the numerological meaning of that particular number from the website.~~
- [x] ~~Be sure to include our standard header at the top of your program with name, username, assignment number, purpose and acknowledgements.~~
- [x] ~~The highest level of your program (i.e., no indenting) should **only** contain the following:~~
    - [x] ~~the header~~
    - [x] ~~any import statements~~
    - [x] ~~function definitions~~
    - [x] ~~a call to the `main()` function~~
- [x] ~~Include comments for the logical sections in your code.~~

Taking it further (Optional - not graded)

Your fruitful function(s) are testable; if you give it certain known values, and you know the expected output, you can write tests using the `assert` statement that validates your function(s). Even better, you can write some tests into a test suite **before** you begin coding the function; once your function works, your tests will tell you (more on **Test Driven Development** in future assignments)!

Following the example given in **hw04_making_change.py**, create a test suite similar to `pennies_test_suite()`. You will need the `unittest(did_pass)` function as well, which prints out the results of the test cleanly.

Submission Instructions

Follow the [submission instructions] to submit your work by next Wednesday at 11:55PM.